COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	07/10/26	**DATE SUBMITTED:**	07/15/26
NAME:	Prince Benjamin S. Villaruel	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1 Basic C++ Programming

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create Code That Follows the Basic C++ Code Structure

1. Create a function that will take two numbers and display the sum.

Sections	Answer
Header File Declaration Section	`#include<iostream>`
Global Declaration Section	`int sumOfTwoNum(int num1, int num2);`
Class Declaration and Method Definition Section	N/A

Main Function	`int main(){`
	`  std::cout<<"The sum of the two numbers: "<<sumOfTwoNum(10, 13);`
	`  return 0;`
	`}`

Method Definition	`int sumOfTwoNum(int num1, int num2){`
	`  int sum = num1 + num2;`
	`  return sum;`
	`}`

Output	`PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 1.1>` `The sum of the two numbers: 23`

2. Create a function that will return whether variable A is greater than variable B.

Sections	Answer
Header File Declaration Section	`#include<iostream>`
Global Declaration Section	`int greaterVariable(int varA, int varB);`
Class Declaration and Method Definition Section	N/A

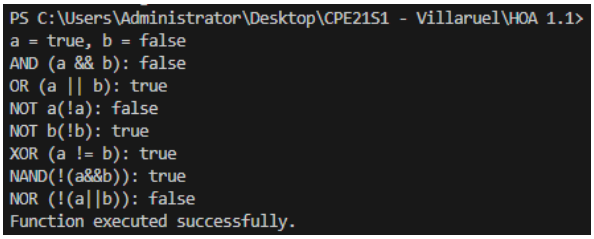
Main Function	`int main(){`
	`  std::cout<<"Variable is A greater than variable B: "<<greaterVariable(6, 7)<<std::endl;`
	`  std::cout<<"Variable is A greater than variable B: "<<greaterVariable(10, 9)<<std::endl;`
	`  return 0;`
	`}`

Method Definition	`int greaterVariable(int varA, int varB){`
	`  if (varA > varB){`
	`  return true;`
	`  }`
	`  else {`
	`  return false;`
	`  }`
	`}`

Output	`PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 1.1>`
	`Variable is A greater than variable B: 0`
	`Variable is A greater than variable B: 1`

3. Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

Sections	Answer
Header File Declaration Section	`#include<iostream>`
Global Declaration Section	N/A
Class Declaration and Method Definition Section	N/A

Main Function	
Method Definition	
Output	

```cpp
int main(){
    std::cout<<std::boolalpha;
    bool result = lops(true, false);
    if(result){
        std::cout<<"Function executed successfully."<<std::endl;
    }
return 0;

}
```

```cpp
bool lops(bool a, bool b){
    std::cout<<"a = "<< a << ", b = "<< b << std::endl;
    std::cout<<"AND (a && b): "<< (a && b)<<std::endl;
    std::cout<<"OR (a || b): "<< (a||b)<<std::endl;
    std::cout<<"NOT a(!a): "<< (!a)<<std::endl;
    std::cout<<"NOT b(!b): "<< (!b)<<std::endl;
    std::cout<<"XOR (a != b): "<< (a != b)<<std::endl;
    std::cout<<"NAND(!(a&&b)): "<< (!(a&&b))<<std::endl;
    std::cout<<"NOR (!(a||b)): "<< (!(a||b))<<std::endl;
    return true;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 1.1>
a = true, b = false
AND (a && b): false
OR (a || b): true
NOT a(!a): false
NOT b(!b): true
XOR (a != b): true
NAND(!(a&&b)): true
NOR (!(a||b)): false
Function executed successfully.
```

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

ILO B output observations and comments.

Triangle.cpp > validateTriangle()

```cpp
#include<iostream>
class Triangle{
    private:
    double totalAngle, angleA, angleB, angleC;
    double sideA, sideB, sideC;


    public:
    Triangle(double A, double B, double C);
    void  setAngles (double A, double B, double C);
    const bool validateTriangle();
};

int main(){
//driver code
    Triangle setl(40, 30, 110);
    if(setl.validateTriangle()){
        std::cout<<"The shape is a valid triangle.\n";
    }
    else {
        std::cout<<"The shape is NOT a valid triangle.\n";
    }
    return 0;
}

Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
const bool Triangle::validateTriangle(){
    return (totalAngle <= 180);
}
```

```
PS C:\Users\Administrator\Desktop\CPE215S1 - Villaruel\HOA 1.1>
The shape is a valid triangle.
```

When I ran this program with the given values (30, 40, 110) provided in the instructions, the output displayed "The shape is a valid triangle." This is right because the three angles add up to exactly 180 degrees, which satisfies the condition in the validateTriangle() function. Looking at the code, I saw that the constructor takes in the three angles when the Triangle object is created and stores them in private variables, then adds them together to get the totalAngle. This part of the Hands-on-Activity showed me how the different parts of the class work together to process and acquire the desired output or result of the program.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

1. Create a C++ program to swap the two numbers in different variables.

Swap.cpp > main()

```cpp
#include<iostream>

int main(){
    int a = 10, b = 5;

    int temp = a;
    a = b;
    b = temp;

    std::cout<< a << " " << b << std::endl;

return 0;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 1.1>
5 10
```

Code Analysis: I wrote this program to swap the values of the two variables, a and b, using a temporary variable. I set a = 10 and b = 5, then used temp to hold a's value before overwriting it. I also learned that there is another method that uses arithmetic (addition and subtraction) to swap values, though the temp method is simpler to understand.

2. Create a C++ program that has a function to convert temperature in Kelvin to Fahrenheit.

Temperature.cpp > main()

```cpp
#include<iostream>
float klvTofhr(float klv){
    float fhr = (klv - 273.15) * 9.0 / 5.0 + 32;
    return fhr;
}

int main(){
    float klv;
    std::cout<<"Kelvin: ";
    std::cin>>klv;

    float result = klvTofhr(klv);
    std::cout<<klv<<" Kelvin is equal to "<<result<<" Fahrenheit"<<std::endl;

    return 0;
}

```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 1.1>
Kelvin: 300
300 Kelvin is equal to 80.33 Fahrenheit
```

Code Analysis: In this program, I wrote a function called klvTofhr that converts a temperature from Kelvin to Fahrenheit using the formula (klv - 273.15) * 9.0 / 5.0 + 32. In main, I simply added a user input and passed it to the function and displayed the result.

3. Create a C++ program that has a function that will calculate the distance between two points.

Distance.cpp > main()

```cpp
#include<iostream>
#include<cmath>

float dst(float x1, float x2, float y1, float y2){
    float dst = sqrt(pow(x2 - x1, 2)+ pow(y2 - y1, 2));
    return dst;
}
int main(){
    std::cout<<"Distance: "<<dst(10, 11, 12, 13)<<std::endl;

    return 0;
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 1.1>
Distance: 1.41421
```

Code Analysis: In this program, I wrote another function called dst that calculates the distance between two points using the distance formula, sqrt(pow(x2 - x1, 2) + pow(pow(y2 - y1, 2)), with the cmath library for the sqrt and pow functions.

4. Modify the code given in ILO B and add the following functions:

a. A function to compute for the area of a triangle
b. A function to compute for the perimeter of a triangle
c. A function that determines whether the triangle is acute-angled, obtuse-angled or 'others.'

Triangle2.cpp > ...

```cpp
#include<iostream>
#include<cmath>
class Triangle{
    private:
    double totalAngle, angleA, angleB, angleC;
    double sideA, sideB, sideC;
    public:
    Triangle(double A, double B, double C);
    void  setAngles (double A, double B, double C);
    void setSides (double a, double b, double c);
    const bool validateTriangle();
    double Area();
    double Perimeter();
    const char* Classification();

};

int main(){
//driver code
    Triangle set1(30, 20, 130);
    set1.setSides(4, 3, 6);
    if(set1.validateTriangle()){
        std::cout<<"The shape is a valid triangle.\n";
        std::cout<<"Perimeter: "<<set1.Perimeter()<<"\n";
        std::cout<<"Area: "<<set1.Area()<<"\n";
        std::cout<<"Classification: "<<set1.Classification()<<"\n";
    }
    else {
        std::cout<<"The shape is NOT a valid triangle.\n";
    }
    return 0;
}

Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
void Triangle::setSides(double a, double b, double c){
    sideA = a;
    sideB = b;
    sideC = c;
}
const bool Triangle::validateTriangle(){
```

```cpp
const bool Triangle::validateTriangle(){
    return (totalAngle <= 180);
}
double Triangle::Perimeter(){
    return sideA + sideB + sideC;
}
double Triangle::Area(){
    double s = (sideA + sideB + sideC) / 2;
    return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
}
const char* Triangle::Classification(){
    double largest = angleA;
    if (angleB > largest) largest = angleB;
    if (angleC > largest) largest = angleC;
    if (largest > 90){
        return "Obtuese-angled";
    }
    else if (largest < 90){
        return "Acute-angled";
    }
    else {
        return "Others";
    }
}
```

```
PS C:\Users\Administrator\Desktop\CPE21S1 - Villaruel\HOA 1.1>
The shape is a valid triangle.
Perimeter: 13
Area: 5.33268
Classification: Obtuese-angled
```

Code Analysis: In the last part of the supplementary activity, I modified the Triangle class in ILO B to add three new functions, which are Area, Perimeter, and Classification. I used Heron's formula for area, simple addition of three sides for perimeter, and a comparison of the three angles to determine the largest one for classification. I tested it with angles 30, 20,130 and sides 4, 3, 6, and the program correctly identified it.

Conclusion

Through this activity, I was able to regain and improve my knowledge of C++ while also learning some new concepts, including how to use and create functions and classes in the next level, declare function prototypes,define member functions, and manipulate variables. Going through each part has helped me recall common mistakes, such as forgetting semicolons, using undeclared variables, or mismatching functions, and debugging errors one at a time makes it easier to fix the rest. I think I did well in this activity despite having limited knowledge when it comes to C++. My main room for improvement is being more analytical and paying closer attention to small details, especially those that users may not always notice but can affect how the program runs overall.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used grammarly to fix and improve my grammar in this activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.